CS 61A
Past Exam Practice — Linked Lists and Trees

Your Name: _____

Your Student ID: _____

Room you are taking the exam in: _____

Name and SID of the person to your **left**: _____

Name and SID of the person to your **right**: _____

You have 110 minutes. There are 6 questions of varying credit. (43 points total)

Question	1	2	3	4	5	6	Total
Points	12	5	4	11	6	5	43

For questions with **circular bubbles**, you may select only one choice.

- ○ Unselected option (Completely unfilled)
- ◌ Don't do this (it will be graded as incorrect)
- ● Only one selected option (completely filled)

For questions with **square checkboxes**, you may select one or more choices.

- ■ You can select
- ■ multiple squares
- ☑ (Don't do this)

Make sure to **write your student ID at the top of each page** in the provided blank.

Anything you write outside the answer boxes or you ~~cross out~~ will not be graded. If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade the worst interpretation. For coding questions with blanks, you may write at most one statement per blank and you may not use more blanks than provided.

As a member of the UC Berkeley community, I act with honesty, integrity, and respect for others. I will follow the rules of this exam. I will not use any other unauthorized materials or devices during this exam. I will not communicate with anyone other than the proctors during this exam. I will not share any information about the exam with anyone until after the end of the exam period.

Acknowledge that you have read and agree to the honor code above and **sign your name below**:

This content is protected and may not be shared, uploaded, or distributed.

Reference: New `Link` and `Tree` Types

Use these definitions for every question in this worksheet.

```
from __future__ import annotations
from dataclasses import dataclass, field

type LinkedList[T] = Link[T] | tuple[()]

@dataclass
class Link[T]:
    first: T
    rest: LinkedList[T] = ()

    def __str__(self):
        return format_link(self)

def format_link(s: Link):
    string = '(' + str(s.first)
    remaining = s.rest
    while isinstance(remaining, Link):
        string += ' ' + str(remaining.first)
        remaining = remaining.rest
    assert remaining == (), f'{s!r} is not a LinkedList'
    return string + ')'

@dataclass
class Tree[T]:
    label: T
    branches: list[Tree[T]] = field(default_factory=list)

    def __str__(self):
        return format_tree(self)

def format_tree(t: Tree, indent='') -> str:
    assert isinstance(t, Tree), f'{t!r} is not a Tree'
    assert isinstance(t.branches, list), f'branches of {t!r} is not a list'
    string = indent + str(t.label)
    for b in t.branches:
        string += '\n' + format_tree(b, indent + '  ')
    return string

def is_leaf(t: Tree) -> bool:
    return not t.branches
```


This content is protected and may not be shared, uploaded, or distributed.

Q1 *All Treeils Lead to Rome* **(12 points)**

Original source: Summer 2023 - Midterm - Question 6

A **treeil** is a tree in which exactly one child of every non-leaf node has a label equal to the label of its parent. Equivalently, from every node there is exactly one path to a leaf consisting entirely of nodes with that same label.

```
1
  1
    1
  2
  4
    4
    5
```

Treeil Trial

Implement `is_treeil`, which determines whether a tree is a treeil.

```
def is_treeil(t):
    """Return whether t is a treeil.

    >>> t1 = Tree(1)
    >>> is_treeil(t1)
    True
    >>> t2 = Tree(1, [Tree(2), Tree(3)])
    >>> is_treeil(t2)
    False
    >>> t3 = Tree(1, [Tree(1, [Tree(1), Tree(3)]), Tree(2)])
    >>> is_treeil(t3)
    True
    >>> t4 = Tree(1, [Tree(1, [Tree(1), Tree(1)]), Tree(2)])
    >>> is_treeil(t4)
    False
    >>> t5 = Tree(2, [Tree(3, [Tree(1)]), Tree(2, [Tree(2)])])
    >>> is_treeil(t5)
    False
    """
    if _________:
       # (a)
        return True
    else:
        match_one = _________([b for b in t.branches if _________]) == 1
                    # (b)                               # (c)
        return _________ and _________([_________ for b in t.branches])
               # (d)       # (e)      # (f)
```


This content is protected and may not be shared, uploaded, or distributed.

Q1.1 (1 point) Fill in blank (a). Select all expressions that work.

- [ ] `True`
- [ ] `False`
- [ ] `is_leaf(t)`
- [ ] `t.branches`
- [ ] `not t.branches`
- [ ] `t.label`
- [ ] `not t.label`
- [ ] `t.branches == []`

Q1.2 (1 point) Fill in blank (b).

- ◯ `sum`
- ◯ `max`
- ◯ `min`
- ◯ `len`
- ◯ `any`
- ◯ `all`

Q1.3 (2 points) Fill in blank (c).

Q1.4 (1 point) Fill in blank (d).

Q1.5 (1 point) Fill in blank (e).

This content is protected and may not be shared, uploaded, or distributed.

- ◯ `sum`
- ◯ `max`
- ◯ `min`
- ◯ `len`
- ◯ `any`
- ◯ `all`

Q1.6 (1 point) Fill in blank (f).

Treeil Trail

Implement `max_treeil`, which returns the largest number of nodes in any treeil that is a subtree of `t`. Assume `is_treeil` is implemented correctly.

```
1
  2
    1
  4  <- root of the largest treeil
    4
      4
    5
```


This content is protected and may not be shared, uploaded, or distributed.

```
def max_treeil(t):
    """Return the highest number of nodes in a treeil within t.

    >>> max_treeil(Tree(1))
    1
    >>> max_treeil(Tree(1, [Tree(2), Tree(3)]))
    1
    >>> t3 = Tree(1, [Tree(1, [Tree(1), Tree(3)]), Tree(2)])
    >>> max_treeil(t3)
    5
    >>> t4 = Tree(1, [Tree(1, [Tree(1), Tree(1)]), Tree(2)])
    >>> max_treeil(t4)
    1
    >>> t5 = Tree(2, [Tree(3, [Tree(1)]), Tree(2, [Tree(2)])])
    >>> max_treeil(t5)
    2
    """
    if _________:
       # (a)
        return 1
    elif _________:
         # (b)
        return 1 + _________
                   # (c)
    else:
        return max([_________ for b in t.branches])
                    # (d)
```

Q1.7 (1 point) Fill in blank (a).

- ◯ `is_treeil(t)`
- ◯ `not is_treeil(t)`
- ◯ `is_leaf(t)`
- ◯ `is_treeil(t) or is_leaf(t)`
- ◯ `not is_treeil(t) and is_leaf(t)`

Q1.8 (1.5 points) Fill in blank (b).

Q1.9 (1 point) Fill in blank (c).

This content is protected and may not be shared, uploaded, or distributed.

(Question 1 continued…)

- ◯ `max([max_treeil(b) for b in t.branches])`
- ◯ `len([max_treeil(b) for b in t.branches])`
- ◯ `sum([max_treeil(b) for b in t.branches])`
- ◯ `sum([b for b in t.branches if is_treeil(b)])`
- ◯ `len([b for b in t.branches if is_treeil(b)])`

Q1.10 (1.5 points) Fill in blank (d).

This content is protected and may not be shared, uploaded, or distributed.

Q2 *Prefixes* **(5 points)**

Original source: Spring 2023 - Midterm 2 - Question 3

Tens

Implement `tens`, which takes a non-empty linked list of numbers `s` represented as a `Link`. It prints all prefix sums of `s` that are multiples of 10, in increasing order of prefix length.

```
def tens(s):
    """Print all prefix sums of Link s that are multiples of ten.

    >>> tens(Link(3, Link(9, Link(8, Link(10, Link(0, Link(14, Link(6)))))))
    20
    30
    30
    50
    """
    def f(suffix, total):
        if total % 10 == 0:
            print(total)
        if _________:
            # (d)
            _________
            # (e)

    _________
    # (f)
```

Q2.1 (1 point) Fill in blank (d).

- ◯ `isinstance(s, Link)`
- ◯ `isinstance(s.rest, Link)`
- ◯ `isinstance(s.rest.rest, Link)`
- ◯ `isinstance(suffix, Link)`
- ◯ `isinstance(suffix.rest, Link)`
- ◯ `isinstance(suffix.rest.rest, Link)`

Q2.2 (2 points) Fill in blank (e).

This content is protected and may not be shared, uploaded, or distributed.

(Question 2 continued…)

Q2.3 (2 points) Fill in blank (f).

This content is protected and may not be shared, uploaded, or distributed.

Q3 *Trees* **(4 points)**

Original source: Spring 2024 - Final - Question 5(a)

Implement `count_ones`, which takes a `Tree` instance `t` and returns the number of nodes in `t` that have exactly one child.

```
def count_ones(t):
    """Return the number of nodes that have one child.

    >>> count_ones(Tree(1, [Tree(2), Tree(3, [Tree(4)])]))
    1
    >>> count_ones(Tree(1, [Tree(2), Tree(3), Tree(4)]))
    0
    >>> count_ones(Tree(1, [Tree(2, [Tree(3, [Tree(4)]),
    ...                               Tree(5, [Tree(6)])])]))
    3
    """
    count = sum([count_ones(_______) for b in t.branches if _______])
                            # (a)                    # (b)
    if _______:
       # (c)
        return count + 1
    else:
        return count
```

Q3.1 (1 point) Fill in blank (a).

- ◯ `b`
- ◯ `t`
- ◯ `b.branches`
- ◯ `t.branches`

Q3.2 (1 point) Fill in blank (b).

- ◯ `True`
- ◯ `is_leaf(b)`
- ◯ `is_leaf(t)`
- ◯ `len(b.branches) == 1`
- ◯ `len(t.branches) == 1`

Q3.3 (2 points) Fill in blank (c).

This content is protected and may not be shared, uploaded, or distributed.

(Question 3 continued…)

- ◯ `True`
- ◯ `is_leaf(b)`
- ◯ `is_leaf(t)`
- ◯ `len(b.branches) == 1`
- ◯ `len(t.branches) == 1`
- ◯ `all([len(b.branches) == 1 for b in t.branches])`
- ◯ `any([len(b.branches) == 1 for b in t.branches])`

This content is protected and may not be shared, uploaded, or distributed.

Q4 *How Long Is This Exam?* **(11 points)**

Original source: Fall 2024 - Midterm 2 - Question 5

Longer

Implement `longer`, which takes two linked lists of numbers `s` and `t`. It returns the longer list. If they have the same length, it returns `s`.

```
def longer(s, t):
    """Return the longer linked list, s or t. (Same length? Return s.)

    >>> print(longer(Link(2, Link(3)), ()))
    (2 3)
    >>> print(longer(Link(2, Link(3)), Link(4, Link(5))))
    (2 3)
    >>> print(longer(Link(2, Link(3)), Link(4, Link(5, Link(6, Link(7))))))
    (4 5 6 7)
    >>> longer((), ()) == ()
    True
    """
    a, b = s, t
    while isinstance(b, Link):
        if _______:
           # (a)
            return _______
                   # (b)
        _______
        # (c)
    return _______
           # (d)
```

Q4.1 (2 points) Fill in blank (a).

Q4.2 (1 point) Fill in blank (b).

- ◯ `a`
- ◯ `b`
- ◯ `s`
- ◯ `t`
- ◯ `longer(a, b)`

This content is protected and may not be shared, uploaded, or distributed.

(Question 4 continued…)

- ◯ `longer(s, t)`

Q4.3 (1 point) Fill in blank (c).

- ◯ `a = a.rest`
- ◯ `a = s.rest`
- ◯ `b = b.rest`
- ◯ `b = t.rest`
- ◯ `a, b = a.rest, b.rest`
- ◯ `a, b = s.rest, t.rest`

Q4.4 (1 point) Fill in blank (d).

- ◯ `a`
- ◯ `b`
- ◯ `s`
- ◯ `t`
- ◯ `longer(a, b)`
- ◯ `longer(s, t)`

Longest

Implement `longest`, which takes a linked list of positive integers `s` and a positive integer `n`. It returns the longest sublist of `s` whose elements sum to at most `n`. Do not mutate `s`. In a tie, return any longest valid sublist. Assume `longer` is implemented correctly.

A sublist contains some, none, or all elements of `s` in their original order. The sum of `()` is 0, and `()` is a sublist of every linked list.

This content is protected and may not be shared, uploaded, or distributed.

(Question 4 continued…)

```
def longest(s, n):
    """Return the longest sublist of s that sums to n or less.

    >>> print(longest(Link(5, Link(1, Link(3, Link(4, Link(2, Link(7)))))), 7))
    (1 3 2)
    >>> print(longest(Link(5, Link(1, Link(3, Link(4, Link(2, Link(7)))))), 70))
    (5 1 3 4 2 7)
    >>> longest(Link(3, Link(4, Link(5))), 2) == ()
    True
    """
    if not isinstance(s, Link):
        return s
    t = _______
        # (e)
    if _______:
       # (f)
        return longer(_______, t)
                      # (g)
    else:
        return t
```

Q4.5 (1 point) Fill in blank (e).

- ◯ `longest(s, n)`
- ◯ `longest(s, n - s.first)`
- ◯ `longest(s.rest, n)`
- ◯ `longest(s.rest, n - s.first)`

Q4.6 (2 points) Fill in blank (f).

Q4.7 (3 points) Fill in blank (g).

This content is protected and may not be shared, uploaded, or distributed.

Q5 *Maximum Exponen-tree-ation* **(6 points)**

Original source: Summer 2021 - Midterm - Question 4

For a list of integers `values`, define its **maximum exponentiation** to be the largest single number obtainable by repeatedly combining consecutive pairs using exponentiation. For example:

```
(3 ** 1) ** (2 ** 3) = 6561
(3 ** (1 ** 2)) ** 3 = 27
```

The maximum exponentiation of `[3, 1, 2, 3]` is 6561. Its tree is:

```
6561
  3
    3
    1
  8
    2
    3
```

Implement `exp_tree`. If the maximum is `b ** n`, its two branches must be the `exp_tree`s of the sublists used to produce `b` and `n`, respectively. Assume `values` has a unique maximum-exponentiation tree.

This content is protected and may not be shared, uploaded, or distributed.

```
def exp_tree(values):
    """Return the exponential tree made from values with the greatest root label.

    >>> print(exp_tree([5]))
    5
    >>> print(exp_tree([3, 2]))
    9
      3
      2
    >>> print(exp_tree([2, 3, 2]))
    512
      2
      9
        3
        2
    >>> print(exp_tree([3, 1, 2, 3]))
    6561
      3
        3
        1
      8
        2
        3
    """
    if __________:
       # (a)
        return __________
               # (b)
    else:
        def tree_at_split(i):
            base = exp_tree(__________)
                            # (c)
            exponent = exp_tree(__________)
                                # (d)
            return Tree(__________, [base, exponent])
                        # (e)
        trees = [tree_at_split(i) for i in range(1, len(values))]
        return max(trees, key=__________)
                               # (f)
```

Q5.1 (1 point) Fill in blank (a).

- ◯ `len(values) == 0`
- ◯ `len(values) == 1`
- ◯ `values`

This content is protected and may not be shared, uploaded, or distributed.

- ◯ `len(values) == 2`
- ◯ `values[0] < values[1]`
- ◯ `len(values) <= 2`
- ◯ `len(values) <= 3`

Q5.2 (1 point) Fill in blank (b).

- ◯ `0`
- ◯ `Tree(0)`
- ◯ `values[0]`
- ◯ `Tree(values[0])`
- ◯ `values[1]`
- ◯ `Tree(values[1])`
- ◯ `values[0] ** values[1]`
- ◯ `Tree(values[0] ** values[1])`
- ◯ `values[1] ** values[0]`
- ◯ `Tree(values[1] ** values[0])`

Q5.3 (1 point) Fill in blank (c).

Q5.4 (1 point) Fill in blank (d).

Q5.5 (1 point) Fill in blank (e).

Q5.6 (1 point) Fill in blank (f).

This content is protected and may not be shared, uploaded, or distributed.

(Question 5 continued…)



This content is protected and may not be shared, uploaded, or distributed.

Q6 *Skipping Around* **(5 points)**

Original source: Summer 2021 - Midterm - Question 2(b)

A **skip list** is a sublist of a list such that no two selected elements were adjacent in the original list. For `[5, 6, 8, 2]`, examples include `[5, 8]`, `[5, 2]`, `[6, 2]`, every singleton list, and `[]`. The empty list is always a skip list.

Given a list `int_lst` of unique integers, return all unique skip lists in which the integers are strictly increasing. The order of the returned skip lists does not matter.

```
1  def list_skipper(int_lst):
2      """Return all strictly increasing skip lists of int_lst.
3
4      >>> list_skipper([5, 6, 8, 2])
5      [[5, 8], [5], [6], [8], [2], []]
6      >>> list_skipper([1, 2, 3, 4, 5])
7      [[1, 3, 5], [1, 3], [1, 4], [1, 5], [1], [2, 4], [2, 5],
8       [2], [3, 5], [3], [4], [5], []]
9      >>> list_skipper([])
10     [[]]
11     """
12     if len(int_lst) == 0:
13         return _________________
14                # (a)
15     with_first = ____________________
16                  # (b)
17     without_first = ________________________
18                     # (c)
19     with_first = [_________ for x in with_first
20                   # (d)
21                   if x == [] or ____________]
22                                 # (e)
23     return with_first + without_first
```

Q6.1 (1 point) Fill in blank (a).

Q6.2 (1 point) Fill in blank (b).

- ◯ `list_skipper(int_lst)`
- ◯ `list_skipper(int_lst[1:])`
- ◯ `list_skipper(int_lst[2:])`

This content is protected and may not be shared, uploaded, or distributed.

- ◯ `list_skipper(int_lst[0])`
- ◯ `list_skipper(int_lst[:-1])`

Q6.3 (1 point) Fill in blank (c).

Q6.4 (1 point) Fill in blank (d).

- ◯ `x`
- ◯ `x[0]`
- ◯ `[x[0]] + int_lst`
- ◯ `[int_lst[0]] + x`
- ◯ `int_lst.append(x)`
- ◯ `x.append(int_lst)`

Q6.5 (1 point) Fill in blank (e).

- ◯ `x[0] < int_lst[0]`
- ◯ `x < int_lst[0]`
- ◯ `x[0] > int_lst[0]`
- ◯ `x > int_lst[0]`
- ◯ `x[0] == int_lst[1]`
- ◯ `len(x) < len(int_lst)`

This content is protected and may not be shared, uploaded, or distributed.

(Question 6 continued…)

This content is protected and may not be shared, uploaded, or distributed.